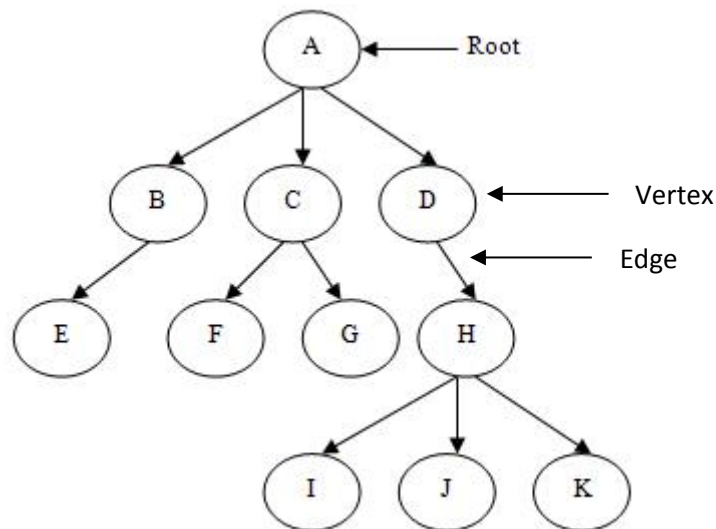


BINARY TREE

Some Definitions

- The circles in a tree diagram are called **vertices** or **nodes**.
- The top of the tree is called its **root**.
- The vertices immediately below a given vertex are called the **children** of that vertex.
- The vertex immediately above a given vertex is called its **parent**. (The root is the only vertex in the tree that has no parent.)
- The line connecting a vertex with one immediately above or below is called a **branch**.
- **Siblings** are vertices with the same parent.
- A vertex with no children is called a **leaf** or an **external** vertex.
- Two branches of a tree are **adjacent** if the lower vertex of the first branch is the upper vertex of the second.
- A sequence of branches in which each is adjacent to its successor is called a **path**.
- The **height** of a tree is the number of vertices on a longest possible path from the root to a leaf. (Hence a tree containing only one vertex has height 1.)
- The **depth** or level of a vertex is the number of branches on a path from the root to the vertex.
- **Forest** is a set of disjoint trees. In other words, collection of more than one trees is called a forest.

Example:



Binary tree: A binary tree T is defined as a set of elements, called nodes, such that:

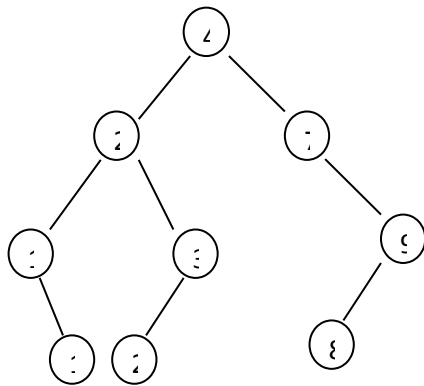
- (a) T is empty (called the null tree or empty tree) OR
- (b) T contains a distinguished node R , called the root node, and the remaining nodes of T form an ordered pair of disjoint binary trees T_1 and T_2

Representation of binary tree: Using array

Binary tree can be stored in an array, say, BT , as follows:

- (a) The root node is stored in $BT[0]$, and
- (b) If a node N occupies $BT[k]$, then its left child is stored in $BT[2*k+1]$ and the right child is stored in $BT[2*k+2]$.

Example:

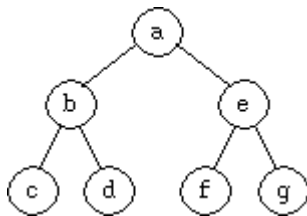


In general, a tree of depth k , requires an array of size 2^{k+1} , approximately.

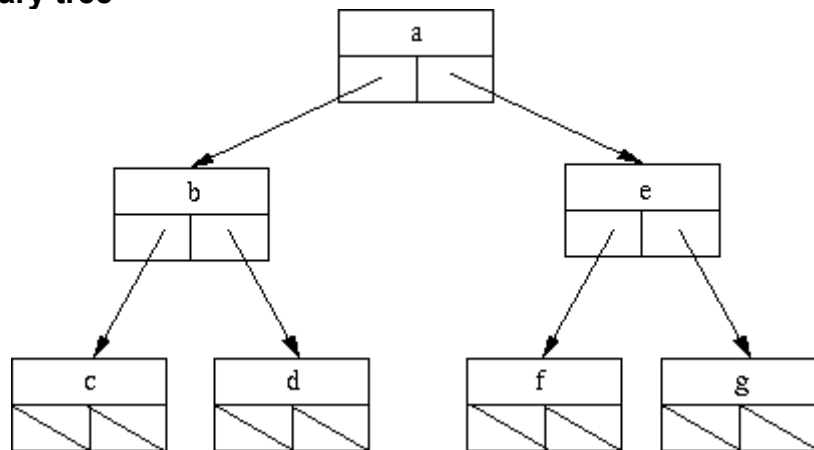
Note: If the root were stored in $BT[1]$, then left child node would occupy $BT[2*k]$ and right child node would occupy $BT[2*k+1]$.

BT	45	22	77	11	30		90		15	25				88						
	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19

Linked representation of binary tree



Binary tree



Linked representation of the binary tree

Structure of the node of a tree:

```
struct node
{
    struct node *left;
    int val;
    struct node *right;
}
```

Binary tree traversal

Preorder traversal

(a) Recursive – Pseudocode

```
Preorder(node *p)
{
    Print p->val;
    Preorder(p->left);
    Preorder(p->right);
}
```

(b) Non-recursive – Algorithm

1. Push *NULL* into *STACK* and set *PTR* = *ROOT*.
2. Repeat steps 3 and 4 while *PTR* ≠ *NULL*
3. Proceed down the left-most path rooted at *PTR*, processing each node *N* on the path and push each right child *R(N)*, if any, into the *STACK*. The traversing ends after a node *N* with no left child *L(N)* is proceeded. (Thus *PTR* is updated using the assignment *PTR* = *LEFT[PTR]* and the traversing stops when *LEFT[PTR]* = *NULL*).
4. Pop and assign to *PTR* the top element of the *STACK*.
5. Exit.

Inorder traversal

(a) Recursive – Pseudocode

```
Inorder(node *p)
{
    Inorder(p->left);
    Print p->val;
    Inorder(p->right);
}
```

(b) Non-recursive – Algorithm

1. Push *NULL* into *STACK* and set *PTR* = *ROOT*.
2. Repeat steps 3 and 4 while *PTR* ≠ *NULL*
3. Proceed down the left-most path rooted at *PTR*, pushing each node *N* into the *STACK*. Stop when a node *N* with no left child is pushed into the *STACK*.
4. Pop and process the node on *STACK*. If *NULL* is popped, then exit. If a node *N* with right child *R(N)* is proceeded, set *PTR* = *R(N)*.
5. Exit.

Postorder traversal

(a) Recursive – Pseudocode

```
Postorder(node *p)
{
    Postorder(p->left);
    Postorder(p->right);
}
```

```

    Print p->val;
}

```

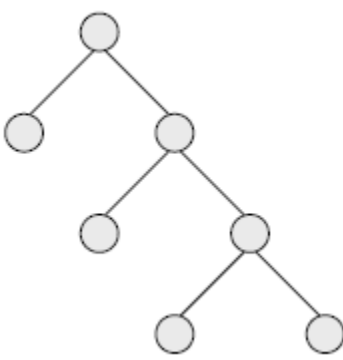
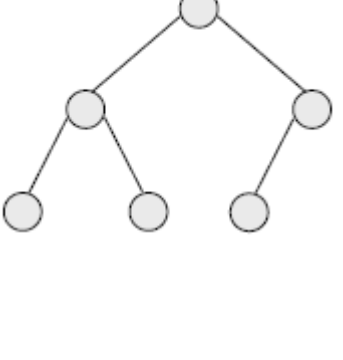
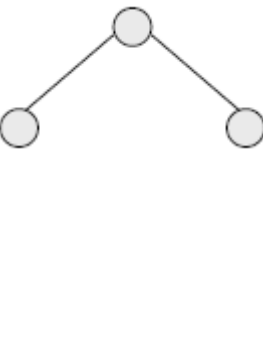
(b) Non-recursive – Algorithm

1. Push *NULL* into *STACK* and set *PTR* = *ROOT*.
2. Repeat steps 3 and 4 while *PTR* ≠ *NULL*
3. Proceed down the left-most path rooted at *PTR*. At each time node *N* of the path push *N* into the *STACK* and if *N* has a right child *R(N)* push *-R(N)* into the *STACK*.
4. Pop and process the positive nodes of *STACK*. If *NULL* is popped, then exit. If a negative node is popped, i.e. if *PTR* = *-N* for some node *N*, set *PTR* = *N* (assuming *PTR* = *-PTR*).
5. Exit.

Full Binary Tree – A binary tree *T* is full if each node is either a leaf or possesses exactly two child nodes.

Complete Binary Tree – A binary tree *T* with *n* levels is complete if all levels except possibly the last are completely full, and the last level has all its nodes to the left side.

Example:

			
Full but not complete	Complete but not full	Neither full nor complete	Full and complete

Theorems:

Let *T* be a nonempty, full binary tree then:

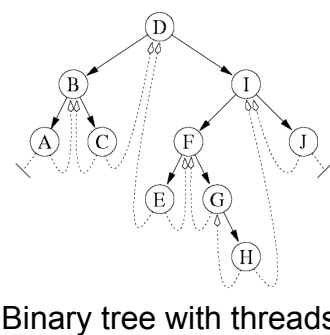
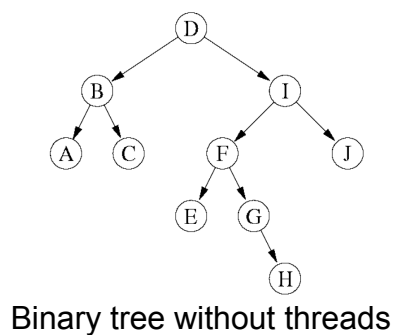
- (a) If *T* has *I* internal nodes, the number of leaves is $L = I + 1$.
- (b) If *T* has *L* leaves, the number of internal nodes is $I = L - 1$.
- (c) If *T* has *I* internal nodes, the total number of nodes is $N = 2I + 1$.
- (d) If *T* has a total of *N* nodes, the number of internal nodes is $I = (N - 1)/2$.
- (e) If *T* has a total of *N* nodes, the number of leaves is $L = (N + 1)/2$.
- (f) If *T* has *L* leaves, the total number of nodes is $N = 2L - 1$.

Theorems:

- (a) Let T be a binary tree with λ levels. Then the number of leaves is at most $2^{\lambda-1}$.
- (b) Let T be a binary tree. For every $k \geq 0$, there are no more than $2k$ nodes in level k .
- (c) Let T be a binary tree with N nodes. Then the number of levels is at least $\lceil \log(N+1) \rceil$.
- (d) Let T be a binary tree with L leaves. Then the number of levels is at least $\lceil \log L \rceil + 1$.

Threaded Binary Tree

A careful observation of the linked representation of any binary tree, it may be noticed that there exist more null links than actual pointers. A clever way to make use of these null links has been devised by A. J. Perlis and C. Thornton. Their idea is to replace the null links by pointers, called threads, to other nodes in the tree. If the $RCHILD(P)$ is normally equal to zero, they suggested to replace it by a pointer to the node which would be printed to after P when traversing the tree in inorder. A null $LCHILD$ link at node P is replaced by a pointer to the node which immediately precedes node P in inorder. Figures below show a binary tree and the corresponding threaded binary tree. Threads are shown as dotted lines.



Note that, left pointer of node A is pointing to null, because, it does not have any predecessor and right pointer of node J is pointing to null, because, it does not have any successor.

If only right threads exit, it is called right in-threaded binary tree.

If only left threads exit, it is called left in-threaded binary tree.

Binary Search Tree (BST)

A **binary search tree (BST)**, is a binary tree data structure which has the following properties:

- The left subtree of a node contains only nodes with keys less than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.

- The left and right subtree must each also be a binary search tree.
- There must be no duplicate nodes.

In BST, time to search for a value is $O(\log n)$, where, n is the number of nodes in the tree.

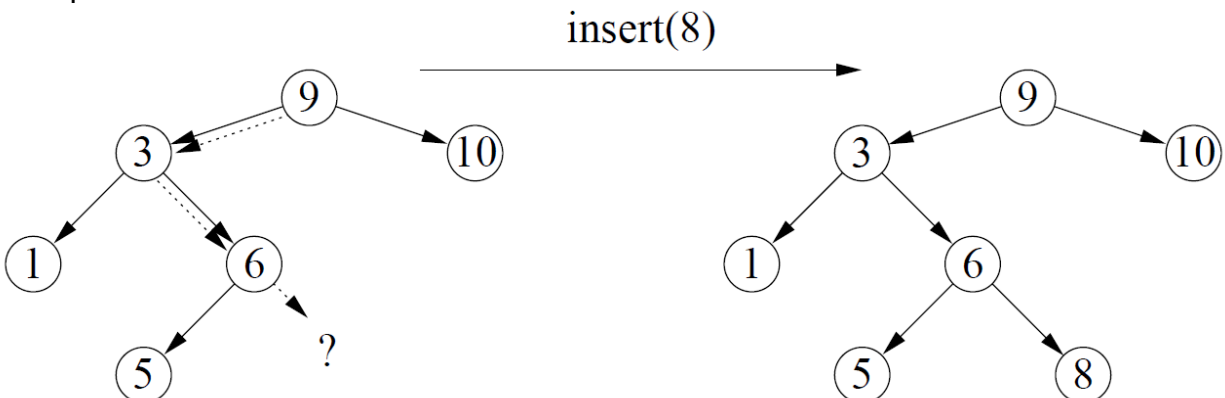
Searching BST

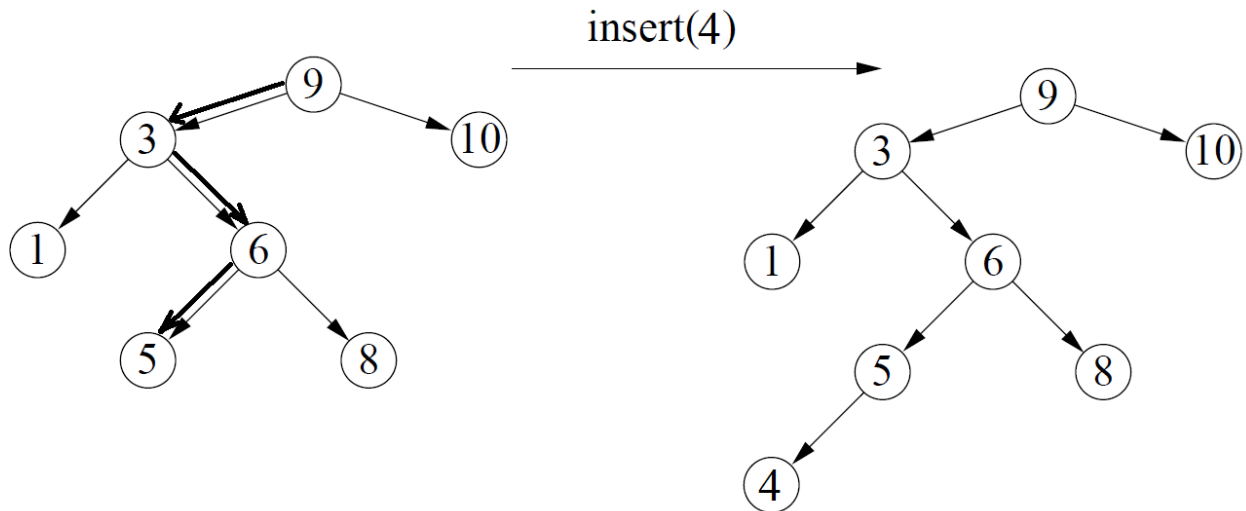
```
algorithm Find-recursive(node, key): node
    node = root
    if node = NULL or node.val = key then
        return node
    else if key < node.val then
        Find-recursive(key, node.left)
    else
        Find-recursive(key, node.right)
```

Inserting a node in a binary search tree

```
insert(x, p) // x is data to insert & p is node
    p = root
    if (p == NULL) // if the tree is empty
        create new node
    else if (x < p.val)
        p.left = insert(x, p.left)
    else if (x > p.val)
        p.right = insert(x, p.right)
    else
        Error: attempt to insert duplicate!
```

Example:





Deleting a node from a binary search tree

Deletion is a more involved procedure than insertion. There are a couple of cases to consider. If the node is a leaf, it can just be deleted with no problem. If it has no left child (or equivalently no right child) then we can just replace the node with its left (or right) child. If both children are present then things are trickier. Removal of the node would create a "hole" in the tree, and we need to fill the hole.

A utility function, `findMin()`, which returns a pointer to the element with the minimum key value in a tree, is required to find the inorder successor. The inorder successor is used to fill the "hole".

```

Algorithm findMin(p):node // assume (p != null)
    while (p.left != NULL)
        p = p.left
    return p

```

```

Algorithm delete(x, p):node
    p = root
    if (p == null) // fell out of tree?
        Error: deletion of nonexistent element!
    else
        if (x < p.val) // x in left subtree
            p.left = delete(x, p.left)
        else if (x > p.data) // x in right subtree
            p.right = delete(x, p.right)
            // => x here, either child empty?
        else if (p.left == null || p.right == null)
            node repl // get replacement
            if (p.left == null) repl = p.right
            if (p.right == null) repl = p.left

```

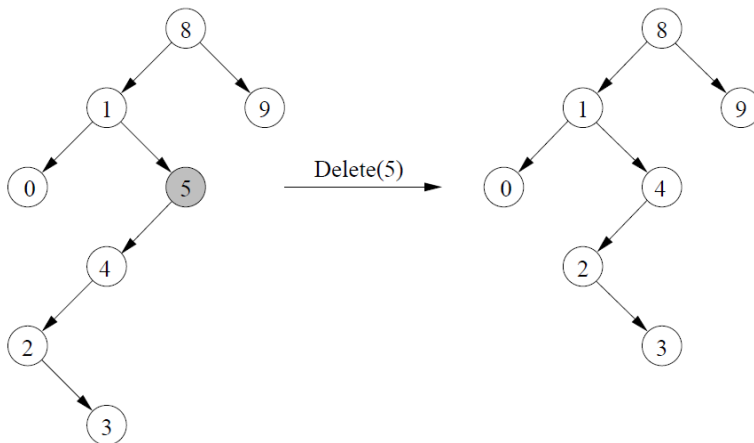
```

    return repl

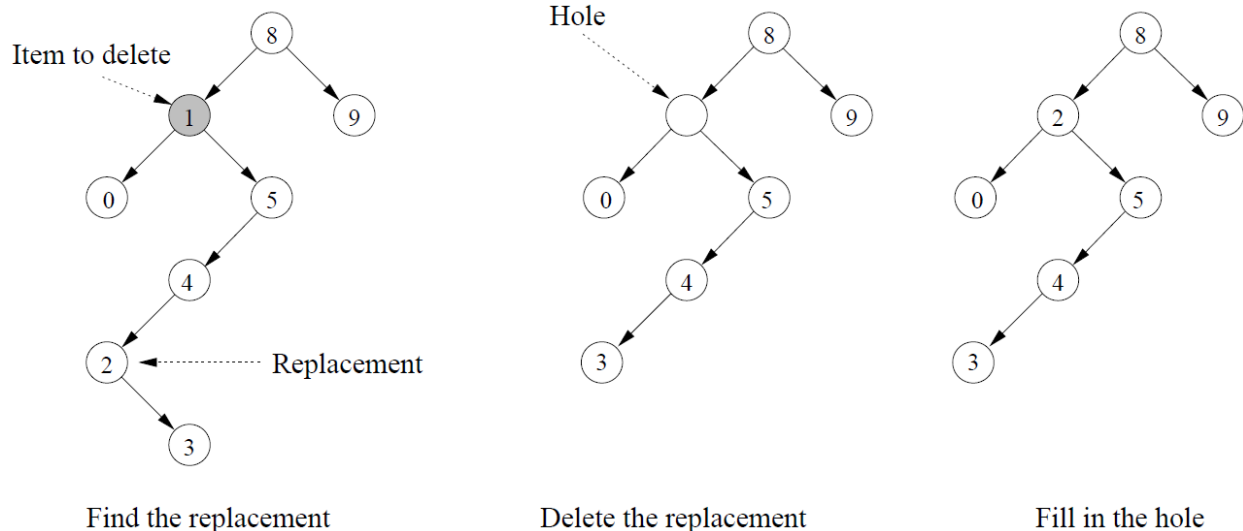
    else // both children present
        p.val = findMin(p.right).val // copy replacement
        p.right = delete(p.val, p.right) // delete replacement
    return p

```

Example 1:



Example 2:



In this context, it should be remembered that the structure of a BST solely depends upon the order in which the data are arranged while constructing the tree. For example:

Insertion order: 9, 3, 10, 6, 1, 8, 5	Insertion order: 8, 9, 5, 10, 3, 6, 1

Analysis of Binary Search Tree

It is not hard to see that all of the procedures `find()`, `insert()`, and `delete()` run in time that is proportional to the height of the tree being considered. (The `delete()` procedure is the only one for which this is not obvious. Note that each recursive call moves us lower in the tree. Also note that `findMin()`, is only called once, since the node that results from this operation does not have a left child, and hence will itself not generate another call to `findMin()`.)

The question is, given a binary search tree T containing n keys, what is the height of the tree? It is not hard to see that in the worst case, if we insert keys in either strictly increasing or strictly decreasing order, then the resulting tree will be completely degenerate, and have height $n - 1$. On the other hand, following the analogy from binary search, if the first key to be inserted is the median element of the set of keys, then it will nicely break the set of keys into two sets of sizes roughly $n/2$ each. This will result in a nicely balanced tree, whose height will be $O(\log n)$. Thus, if you had a million nodes, the worst case height is a million and the best is around 20, which is quite a difference. An interesting question: what is the expected height of the tree, assuming that keys are inserted in random order?

Textbooks give a careful analysis of the average case, but it is based on solving a complex recurrence. Instead, we will present a simple probabilistic analysis to show that the number of nodes along the leftmost chain of the tree is expected to be $\ln(n)$. Since the leftmost path is a representative path in the tree, it is not surprising that the average path length is still $O(\log n)$ (and a detailed analysis shows that the expected path length is actually $2\ln(n)$).